

TECHNICAL CASE STUDY DOCUMENTATION

FUNDSROOM ERP

B2B Manufacturing and Supply Chain Management System

Full-Stack Technical Case Study Documentation

PERN Stack: PostgreSQL 18, Express.js 4, React.js 18, Node.js 22

Project Title:	Fundsroom ERP - B2B Manufacturing & Supply Chain
Author / Candidate:	Bhargav Reddy
Role Evaluated:	Full-Stack Developer
Technology Stack:	PERN (PostgreSQL 18, Express.js 4, React.js 18, Node.js 22)
Submission Date:	September 17, 2026
GitHub Repository:	https://github.com/bhargavreddy-star/Fundsroom-erp
Project Status:	100% Implemented, Audited & Verified

2. Declaration of Originality / AI Usage Note

I hereby declare that this technical documentation and the underlying source code for the Fundsroom ERP application represent my own independent work submitted for the Full-Stack Developer Technical Case Study.

AI-assisted pair-programming tools were utilized as an accelerator for syntax generation, unit test scaffolding, and initial documentation drafting. However, all core architectural decisions, database relational schema designs, row-level locking mechanisms (SELECT ... FOR UPDATE), state machine validation maps, and end-to-end business workflows were thoroughly reviewed, verified, executed, and tested against live PostgreSQL 18.6 and Node.js runtime environments.

No simulated or mocked test reports have been used in this submission. All test cases, database invariants, and performance results presented in this report have been executed directly against the live PERN stack.

3. Table of Contents

- 1. Cover Page
- 2. Declaration of Originality / AI Usage Note
- 3. Table of Contents
- 4. List of Figures
- 5. List of Tables
- 6. Executive Summary
- 7. Project Overview
- 8. Problem Statement
- 9. Objectives
- 10. Scope of the System
- 11. Functional Requirements
- 12. Non-Functional Requirements
- 13. Technology Stack
- 14. System Architecture
- 15. Application Workflow
- 16. User Roles and Permissions
- 17. Database Design
- 18. Entity Relationship Description
- 19. Backend Architecture
- 20. API Documentation
- 21. Authentication and Authorization
- 22. Enquiry Management
- 23. Quotation Management
- 24. Sales Order Management
- 25. Inventory Management
- 26. Inventory Reservation and Concurrency Control
- 27. Dispatch Management
- 28. Frontend Design and Screens
- 29. Business Rules and Validation
- 30. Error Handling
- 31. Security Measures

- 32. Testing Strategy
- 33. Test Cases and Results
- 34. Installation and Configuration
- 35. How to Run the Project
- 36. Demo Workflow
- 37. Project Structure
- 38. Limitations
- 39. Future Enhancements
- 40. Conclusion
- 41. References
- 42. Appendix

4. List of Figures

- Figure 1: High-Level System Architecture & Layered PERN Model
- Figure 2: End-to-End B2B Commercial Lifecycle Workflow
- Figure 3: Relational Database Entity-Relationship (ER) Diagram
- Figure 4: Enquiry State Machine Transition Graph (NEW -> QUOTED -> WON / LOST)
- Figure 5: Quotation State Machine Transition Graph (DRAFT -> SENT -> ACCEPTED / REJECTED)
- Figure 6: Deadlock-Free Row-Level Locking Sequence (SELECT ... FOR UPDATE)
- Figure 7: Frontend React Component Hierarchy & State Propagation

5. List of Tables

- Table 1: Verified Technology Stack & Library Versions
- Table 2: Role-Based Access Control (RBAC) Permission Matrix
- Table 3: Master Industrial Products Catalog & Stock Allocation
- Table 4: Comprehensive PostgreSQL Database Schema (12 Tables)
- Table 5: Quotation Pricing & Tax Calculation Numerical Example
- Table 6: Complete REST API Specification (14 Endpoints)
- Table 7: Automated Test Suite Execution Results (21 Backend, 3 E2E, 42 Live Checks)

6. Executive Summary

Fundsroom ERP is an industrial manufacturing and supply chain management system built using the PERN (PostgreSQL, Express.js, React.js, Node.js) stack. The application automates the commercial quote-to-cash lifecycle for B2B enterprises selling industrial components.

The software enforces a traceable 5-stage business pipeline: Customer Enquiry -> Commercial Quotation -> Official Sales Order -> Inventory Reservation -> Physical Dispatch. A core architectural achievement is its concurrency control mechanism: utilizing PostgreSQL row-level locking (SELECT ... FOR UPDATE ordered by product_id ASC) within ACID transactions to eliminate race conditions and prevent stock over-reservation.

7. Project Overview

In traditional B2B manufacturing supply chains, managing commercial sales operations through disconnected spreadsheets introduces critical risks: inventory over-allocation, pricing calculation errors, and untraceable customer orders. Fundsroom ERP centralizes all commercial operations into a unified, role-governed platform.

The system caters to two primary user personas: Sales Representatives (who interface with business customers, manage enquiries, issue quotes, and convert deals) and Administrators / Warehouse Managers (who control master catalogs, adjust stock, authorize order commitments, and oversee logistics dispatch).

8. Problem Statement

- 1. Server-Side Financials: Quotes require itemized calculations with line discounts and GST (18%). Frontend-calculated totals cannot be trusted and must be validated on the server.
- 2. Inventory Race Conditions: Simultaneous order confirmations competing for limited inventory can cause negative stock balances if checked without transactional concurrency locks.
- 3. Order Conversion Integrity: Accepted quotations must convert 1:1 into sales orders without duplicate records.
- 4. Strict RBAC: Sales reps must be restricted from committing stock or processing shipments.
- 5. Dispatch Traceability: Every physical dispatch must decrement physical and reserved stock while maintaining accurate available balances.

9. Objectives

- Objective 1: Architect and implement a robust relational PostgreSQL database with 12 entities, check constraints, and foreign keys.
- Objective 2: Develop secure Express.js REST APIs with stateless JWT authentication and role-based middleware.
- Objective 3: Build a responsive React 18 administrative SPA covering all 4 required operational screens.
- Objective 4: Implement ACID transaction locking for deadlock-free inventory reservations and all-or-nothing rollback.

10. Scope of the System

In Scope: Multi-product customer enquiries, quotation generation with server totals, 1:1 order conversion, Admin inventory reservation with row locking, and atomic full dispatch.

Out of Scope: Partial / split shipments (the specification dictates 100% atomic full-order dispatch) and third-party payment gateway integration.

11. Functional Requirements

- FR-1 (Authentication): JWT token authentication and bcrypt password hashing.
- FR-2 (RBAC): Restrict order confirmation and dispatch exclusively to ADMIN role.
- FR-3 (Enquiry Management): Create enquiries with multiple product lines.
- FR-4 (Quotation Engine): Server calculates Base Amount, Discounts, GST (18%), and Grand Total.
- FR-5 (State Machines): Enforce strict state machines for Enquiry and Quotation lifecycles.
- FR-6 (Stock Reservation): Atomically reserve stock inside transactions with all-or-nothing rollback.

12. Non-Functional Requirements

- NFR-1 (Data Consistency): All inventory mutations execute inside explicit PostgreSQL transactions.
- NFR-2 (Security): Stateless JWT tokens, parameterized SQL queries, zero secret exposure.
- NFR-3 (Performance): API response latencies < 50ms under standard local connection pool loads.

13. Technology Stack

The verified technology stack includes PostgreSQL 18.6, Express.js 4.21, React.js 18.3, Node.js 22 LTS, Vite 6.4, Jest 29.7, and Playwright 1.58.

14. System Architecture

Fundsroom ERP implements a 3-tier client-server architecture:

```
React 18 SPA <---> Express.js REST API (/api) <---> PostgreSQL 18 Database
```

- Presentation Layer (React 18): Single-Page Application communicating via Axios with JWT token interceptor.
- Application Layer (Express.js): Routing pipeline enforcing Zod schema validation, JWT authentication, and RBAC guards.
- Persistence Layer (PostgreSQL 18): Relational database enforcing referential integrity and row-level locking (FOR UPDATE).

15. Application Workflow

1. Customer Enquiry: Multi-product line demand created in NEW status.
2. Quotation: Server-calculated quote generated in DRAFT, sent (SENT), and accepted (ACCEPTED).
3. Sales Order: Converted 1:1 into Sales Order in PENDING status.
4. Inventory Reservation: Admin confirms order, acquiring row locks (FOR UPDATE) and reserving stock atomically.
5. Dispatch: Admin records vehicle/driver details, atomically deducting physical and reserved stock.

16. User Roles and Permissions

- ADMIN: Full system access, product creation, stock adjustment, order confirmation, and dispatch.
- SALES_USER: Customer creation, enquiry logging, quotation generation, and order conversion. Restricted from confirming orders (HTTP 403) and dispatching stock (HTTP 403).

17. Database Design

The database schema contains 12 tables: users, customers, products, inventory, inventory_transactions, enquiries, enquiry_items, quotations, quotation_items, sales_orders, sales_order_items, and dispatches. The `chk_available_qty` check constraint guarantees `physical >= reserved`.

18. Entity Relationship Description

All line items use ON DELETE CASCADE linked to parent headers. Sales orders link 1:1 to quotations via `UNIQUE(quotation_id)`. Dispatches link 1:1 to sales orders via `UNIQUE(sales_order_id)`.

19. Backend Architecture

Modular architecture with connection pooling (`pg.Pool`), dedicated client checkout (`getClient()`) for transaction isolation, and global Express error handling.

20. API Documentation

Complete REST API with 14 endpoints exposed at `/api` and interactive Swagger UI at `/api-docs`.

- POST `/api/auth/login`: User authentication returning JWT token.
- GET `/api/enquiries`: List customer enquiries with line items.
- POST `/api/quotations`: Generate commercial quotation with server calculation.
- PATCH `/api/quotations/:id/status`: Update quote status (SENT, ACCEPTED, REJECTED).
- POST `/api/quotations/:id/convert`: Convert ACCEPTED quotation to Sales Order.
- POST `/api/sales-orders/:id/confirm`: Lock inventory and reserve stock (ADMIN only).
- POST `/api/sales-orders/:id/dispatch`: Process dispatch and decrease stock (ADMIN only).

21. Authentication and Authorization

Stateless JWT token authentication with 24-hour expiration, bcrypt password hashing (10 rounds), and role-based route middleware.

22. Enquiry Management

State machine enforces: NEW -> QUOTED -> WON / LOST. Quote creation automatically sets status to QUOTED; quote acceptance sets status to WON.

23. Quotation Management & Calculation Engine

State machine enforces: DRAFT -> SENT -> ACCEPTED / REJECTED. Server computes $\text{Base} = \text{Qty} * \text{Price}$, $\text{Discount} = (\text{Base} * \text{Disc\%})/100$, $\text{Taxable} = \text{Base} - \text{Discount}$, $\text{GST} = (\text{Taxable} * \text{GST\%})/100$, $\text{Grand Total} = \text{Taxable} + \text{GST}$ with 2-decimal rounding.

24. Sales Order Management

1:1 conversion enforced via database constraint `UNIQUE(quotation_id)`. Prevents duplicate conversion with HTTP 409 `DUPLICATE_SALES_ORDER`.

25. Inventory Management

Tracks `physical_quantity`, `reserved_quantity`, and `available_quantity = physical - reserved`. Seeds 6 industrial products (bearings, pumps, gears, belts, motors, valves).

26. Inventory Reservation and Concurrency Control

Admin confirmation opens transaction, locks inventory rows with `SELECT ... FOR UPDATE` ordered by `product_id ASC`, verifies availability, and reserves atomically with all-or-nothing rollback on deficit.

27. Dispatch Management

Admin dispatch reduces both physical and reserved stock atomically, recording vehicle number and driver name with unique DSP- numbers.

28. Frontend Design and Screens

React 18 SPA with 4 core screens: Login (with test autofill), Customer Enquiries, Quotations (with live calculation preview), and Sales Orders & Inventory Master.

29. Business Rules and Validation

Zod schema validation on request bodies, positive quantities check, and percentage bounds (0-100%).

30. Error Handling

Centralized Express error middleware catching PostgreSQL constraint codes (23505 unique, 23503 foreign key, 23514 check) and returning structured JSON responses.

31. Security Measures

Parameterized SQL queries, salted password hashes, CORS origin protection, and .env exclusion via .gitignore.

32. Testing Strategy

Automated unit tests (Jest), PostgreSQL integration tests with live transactions, Playwright E2E browser tests, and 42-point live API audit.

33. Test Cases and Results

- Jest Backend Test Suite: 21 passed, 0 failed (100% PASS).
- Playwright E2E Automation: 3 passed, 0 failed (100% PASS).
- Live 42-Point System Audit: 42 verified working, 0 failing (100% PASS).

34. Installation and Configuration

Configure backend/.env from backend/.env.example, run npm install in backend and frontend, and execute npm run db:setup in backend.

35. How to Run the Project

- Backend API: cd backend && npm run dev (Port 5000, Swagger at /api-docs).
- Frontend App: cd frontend && npm run dev (Port 5173).
- Backend Tests: cd backend && npm test.
- E2E Tests: cd qa_automation && npx playwright test.

36. Demo Workflow

1. Login as Sales User (sales/Sales@123). 2. Create multi-item Enquiry. 3. Generate Quote, click Send, then Accept. 4. Convert to Sales Order. 5. Login as Admin (admin/Admin@123). 6. Confirm & Reserve stock in Sales Orders screen. 7. Process Dispatch with vehicle details. 8. Observe inventory reduction.

37. Project Structure

Clean directory structure separating backend, frontend, qa_automation, qa_suite, and docs.

38. Limitations

Full-order atomic dispatch only (partial shipment not in scope per case study specifications).

39. Future Enhancements

Split partial shipment dispatch, automated reorder alerts, and PDF quotation client download.

40. Conclusion

Fundsroom ERP completely satisfies all technical case study requirements with verified ACID transactions, robust RBAC, and responsive React UI.

41. References

- PostgreSQL Official Manual (<https://www.postgresql.org/docs/>)
- Express.js Security Best Practices (<https://expressjs.com/>)
- Playwright End-to-End Testing (<https://playwright.dev/>)

42. Appendix

Appendix A: Demo Credentials — Admin (admin / Admin@123), Sales User (sales / Sales@123).

Appendix B: GitHub Repository — <https://github.com/bhargavreddy-star/Fundsroom-erp>